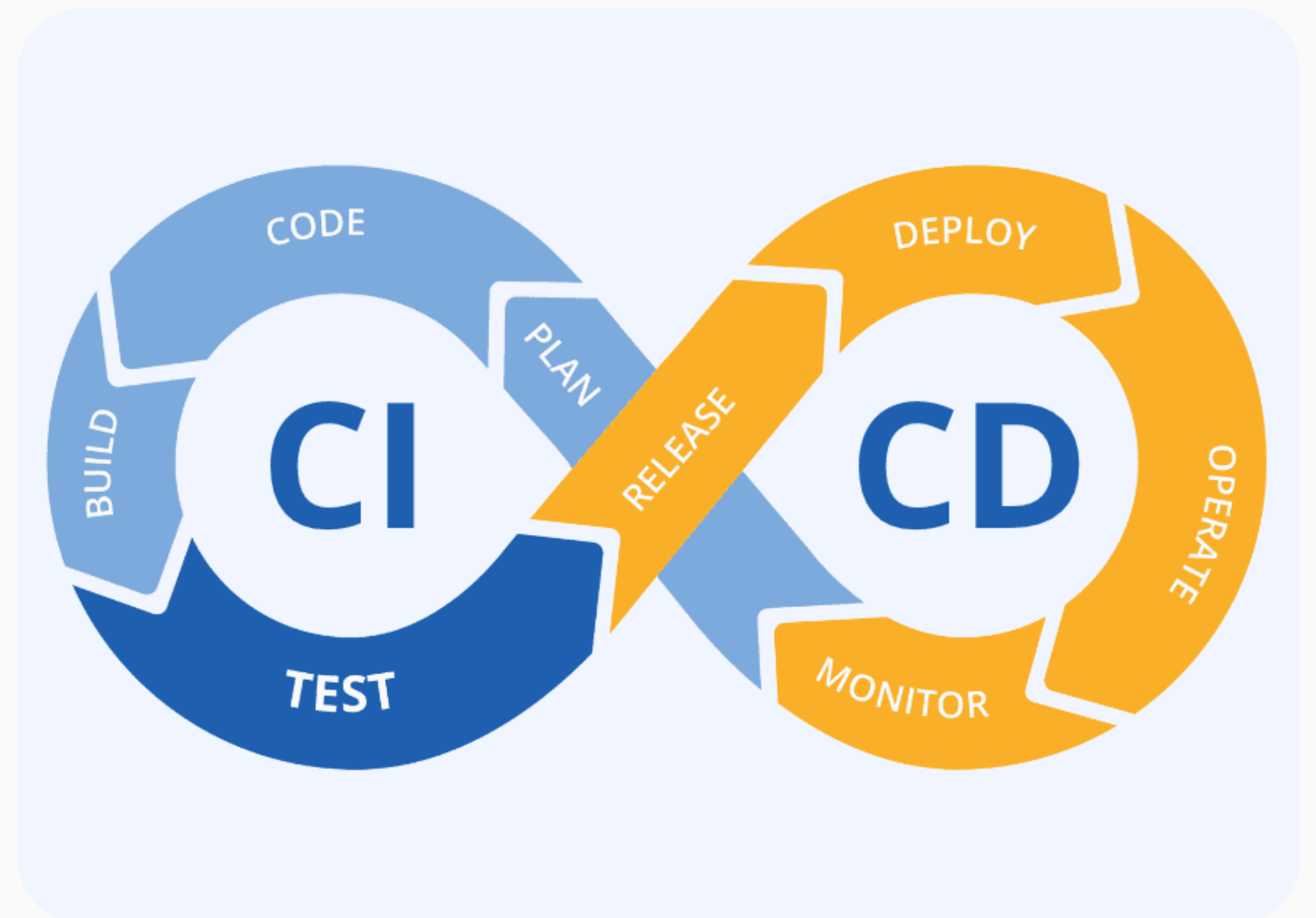


IEC

Integração e Entrega Contínua (CI/CD)

Python



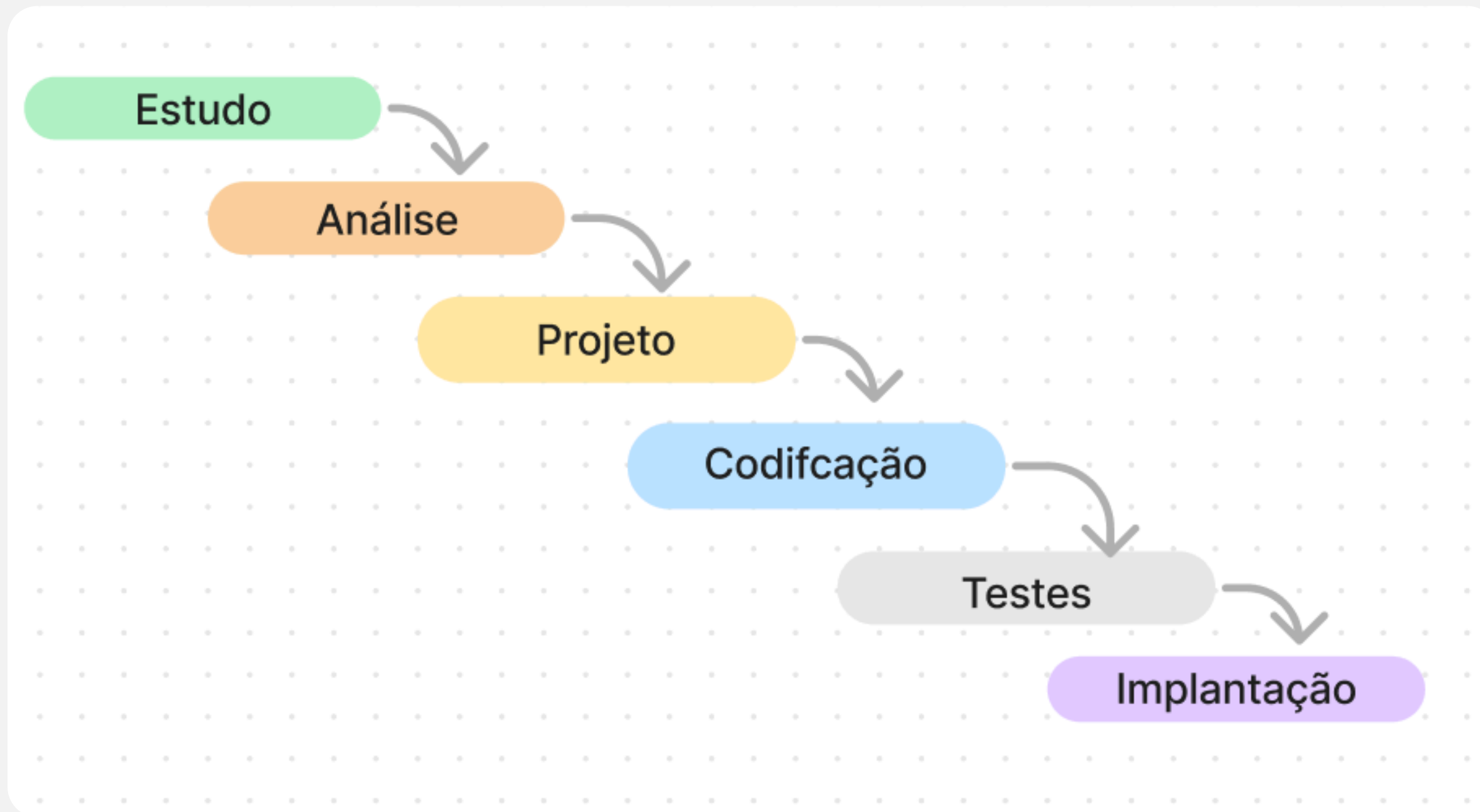
// Prof. Icaro Davies

```
package.json  index.tsx  config.yml
> {..} config.yml > {} workflows
You, 4 months ago | 1 author (You)
version: 2.1

jobs:
  deploy:
    docker:
      - image: circleci/node:18.15.0
    steps:
      - checkout
      - run:
        name: Installing dependencies
        command: npm install
      - run:
        name: Deploying app to prod
```

O Papel dos Testes Automatizados

A Evolução do Desenvolvimento de Software

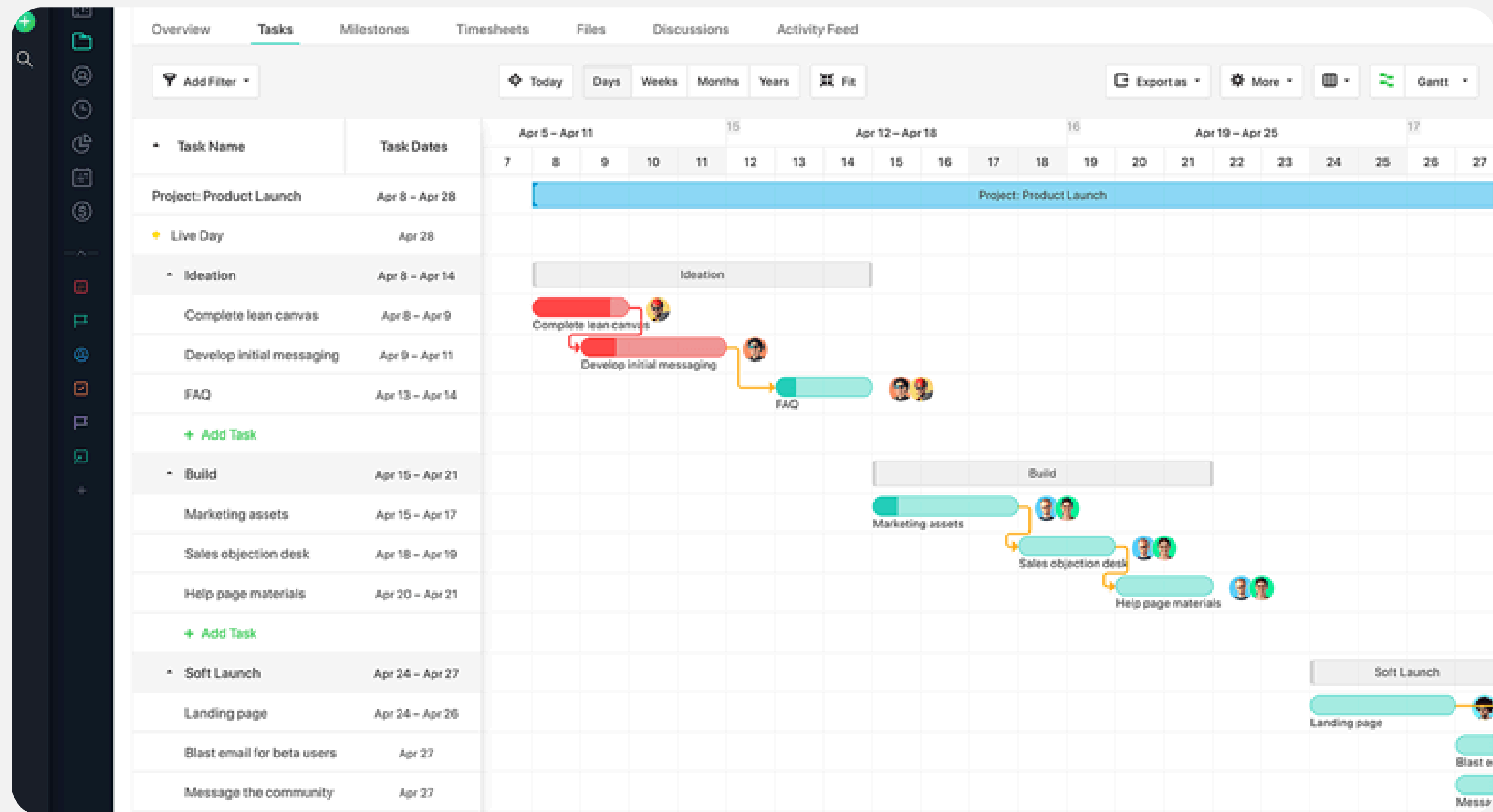


O Modelo Cascata

Caracterizada por uma abordagem linear e sequencial

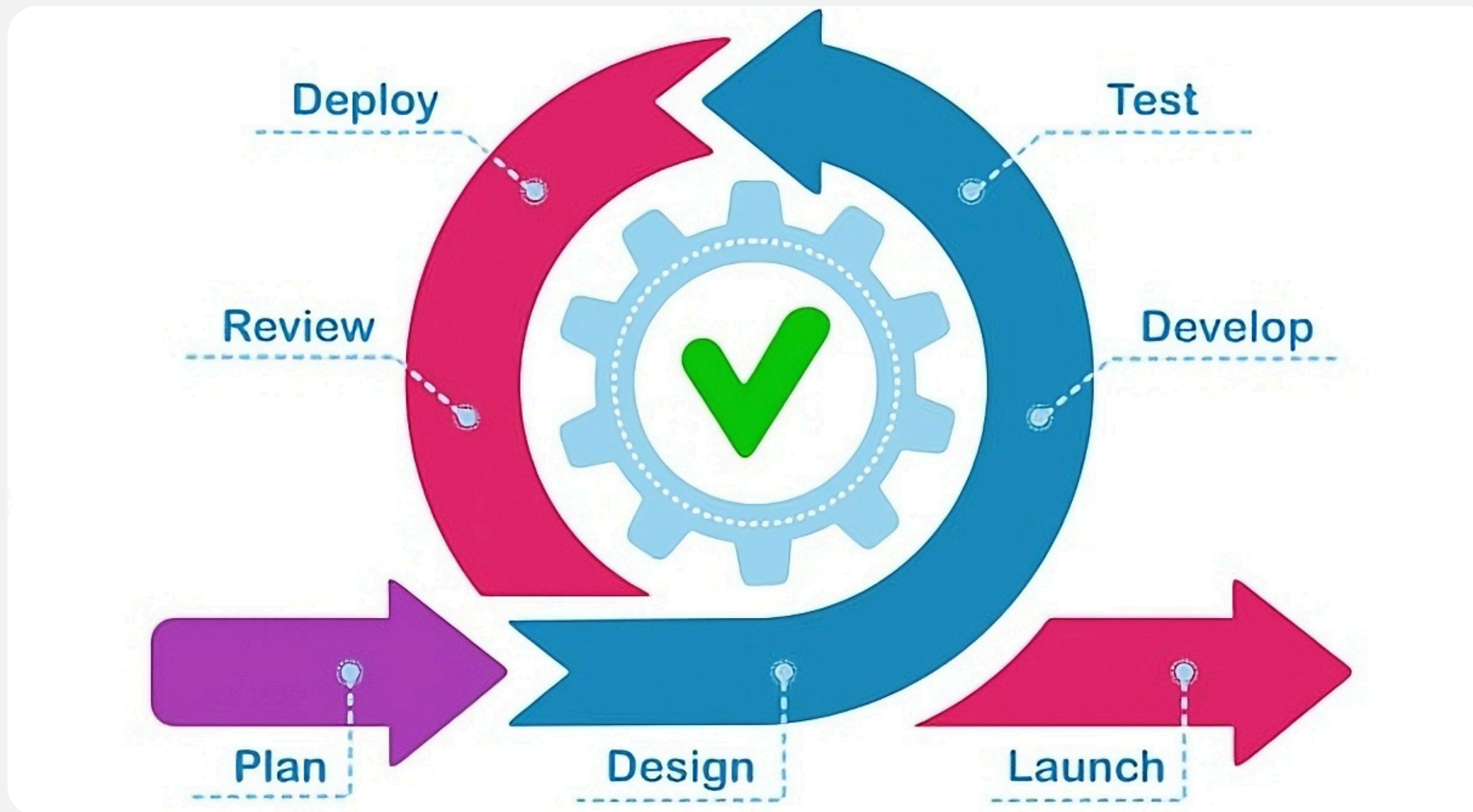


Inspirado em processos de manufatura e construção
organiza o desenvolvimento em etapas **rígidas**



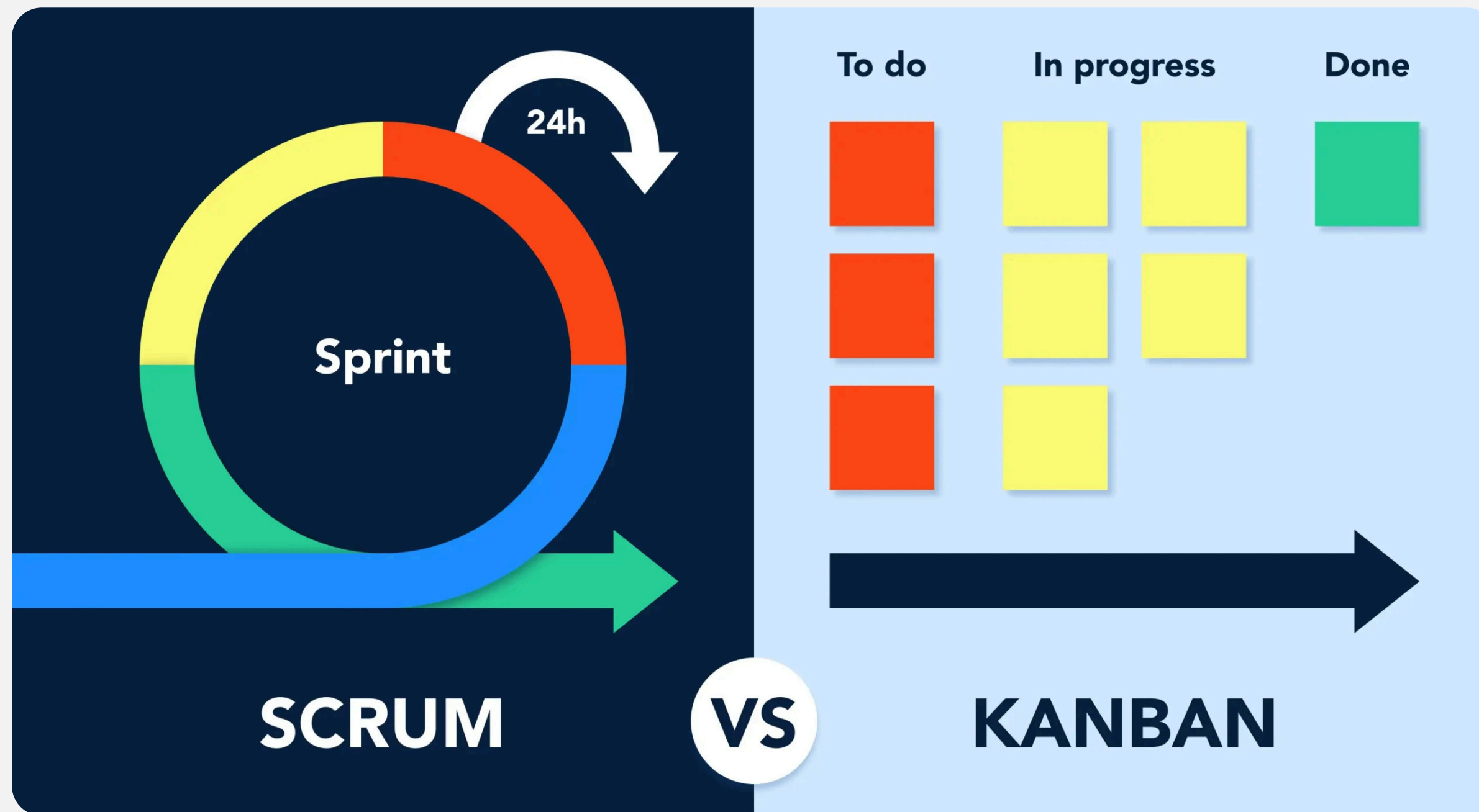
Etapas são distintas e irreversíveis, uma etapa só pode começar após a **conclusão** da anterior

A ascensão do Ágil



O Modelo Ágil

Trocando a rigidez sequencial por um processo iterativo e incremental



O Desenvolvimento é dividido em **ciclos** curtos, ou **melhoria** contínua

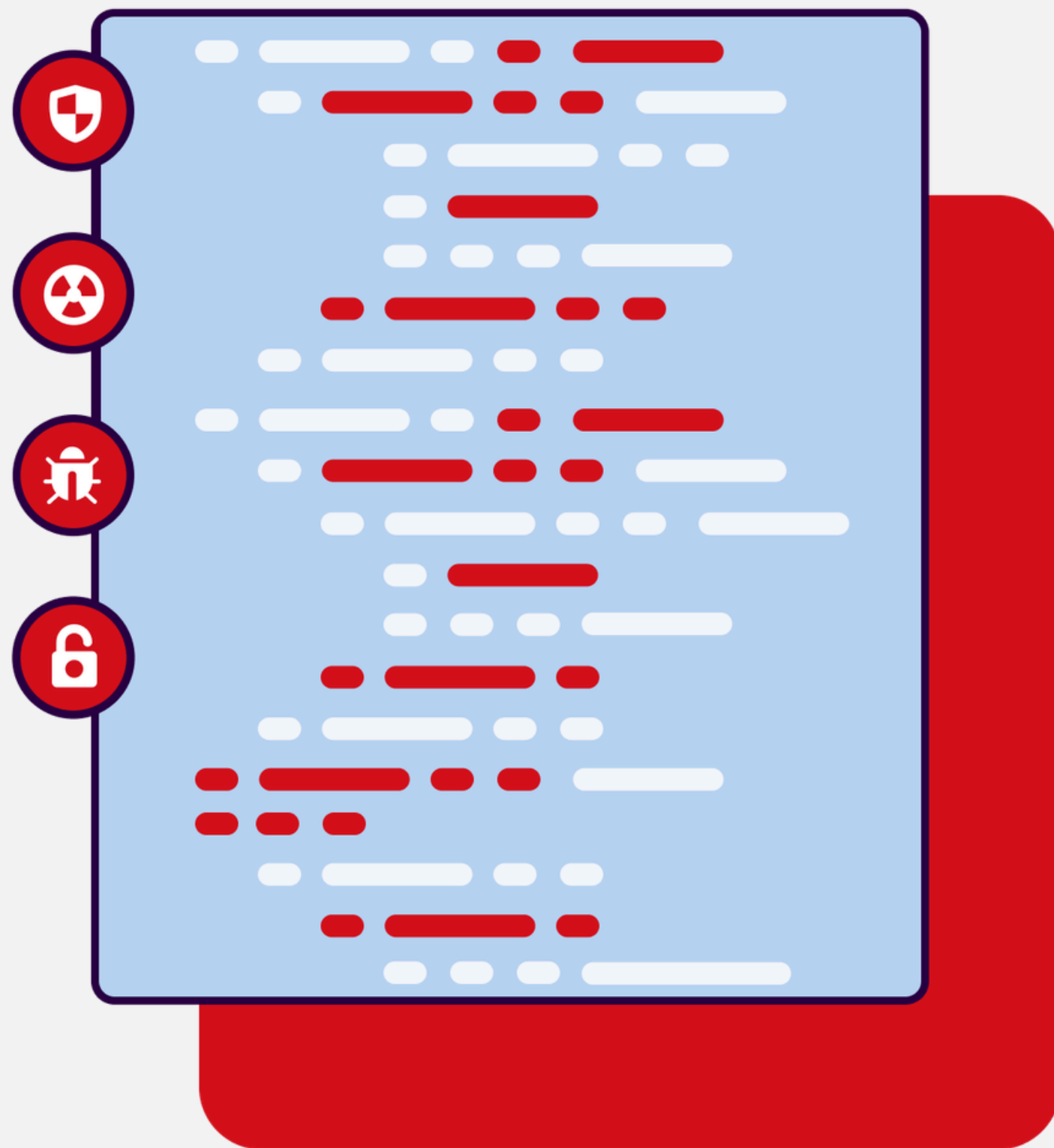


Capacidade de responder a **mudanças** em qualquer estágio do projeto



Colaboração **contínua** com o cliente e entrega frequente de software funcional

Novo desafio técnico

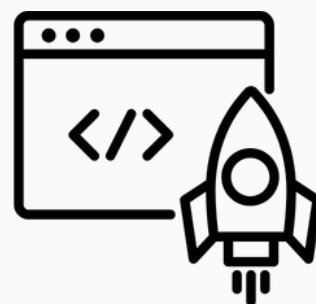


Garantir a
integridade
do código em
cada alteração

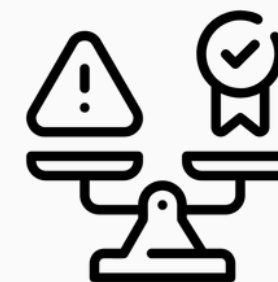
O Cenário Atual do **Desenvolvimento** de Software



Complexidade
Crescente



Pressão por
Velocidade



Qualidade e
Estabilidade

CI / CD

CI / CD

Integração Contínua
Continuous integration

CI / CD

Integração Contínua
Continuous integration

Entrega Contínua
Continuous Delivery

CI / CD

Integração Contínua
Continuous integration

Entrega Contínua
Continuous Delivery

Implantação Contínua
Continuous Deployment

Integração Contínua (CI)

Continuous integration

Integração Contínua (CI)

Princípios Fundamentais :

Repositório Centralizado :

- Github
- Gitlab
- Bitbucket

Build Automatizado :

- Download dependências
- Processo de compilação

Foco:

**Integrar frequentemente
e testar automaticamente**

Saída:

**Código integrado e
testado no repositório
principal**

Integração Contínua (CI)

Testes Automatizados :

- Unitários
- Integração
- Segurança

Feedback Rápido :

- Notificações de falhas

Foco:

**Integrar frequentemente
e testar automaticamente**

Saída:

**Código integrado e
testado no repositório
principal**

Entrega Contínua (CD)

Continuous Delivery

Entrega Contínua (CD)

Princípios Fundamentais :

Dependência :

- Job CI anterior

Ambientes Automatizados :

- Provisionamento
- Configuração de ambientes
- Teste e homologação

Foco:

**Software sempre pronto
para deploy**

Saída:

**Deploy manual em
staging ou produção**

Entrega Contínua (CD)

Testes Abrangentes :

- Integração complexos
- Aceitação de usuário
- Regressão
- Segurança dinâmica
- Performance e carga

Artefatos Gerenciados :

[Docker, Jar, War, APK, IPA]

Foco:

**Software sempre pronto
para deploy**

Saída:

**Deploy manual em
staging ou produção**

Implantação Contínua (CD) Continuous Deployment

Implantação Contínua (CD)

Princípios Fundamentais :

Dependência :

- Job CD anterior

Estratégias de Deploy :

- Blue/Green
- Canary

Monitoramento e Observabilidade :

[Micrometer, Prometheus, Grafana]

Foco:

**Implantação automática
sem intervenção manual**

Saída:

**Entrega rápida e
confiável em produção**

Benefícios e Desafios de CI/CD

// Benefícios

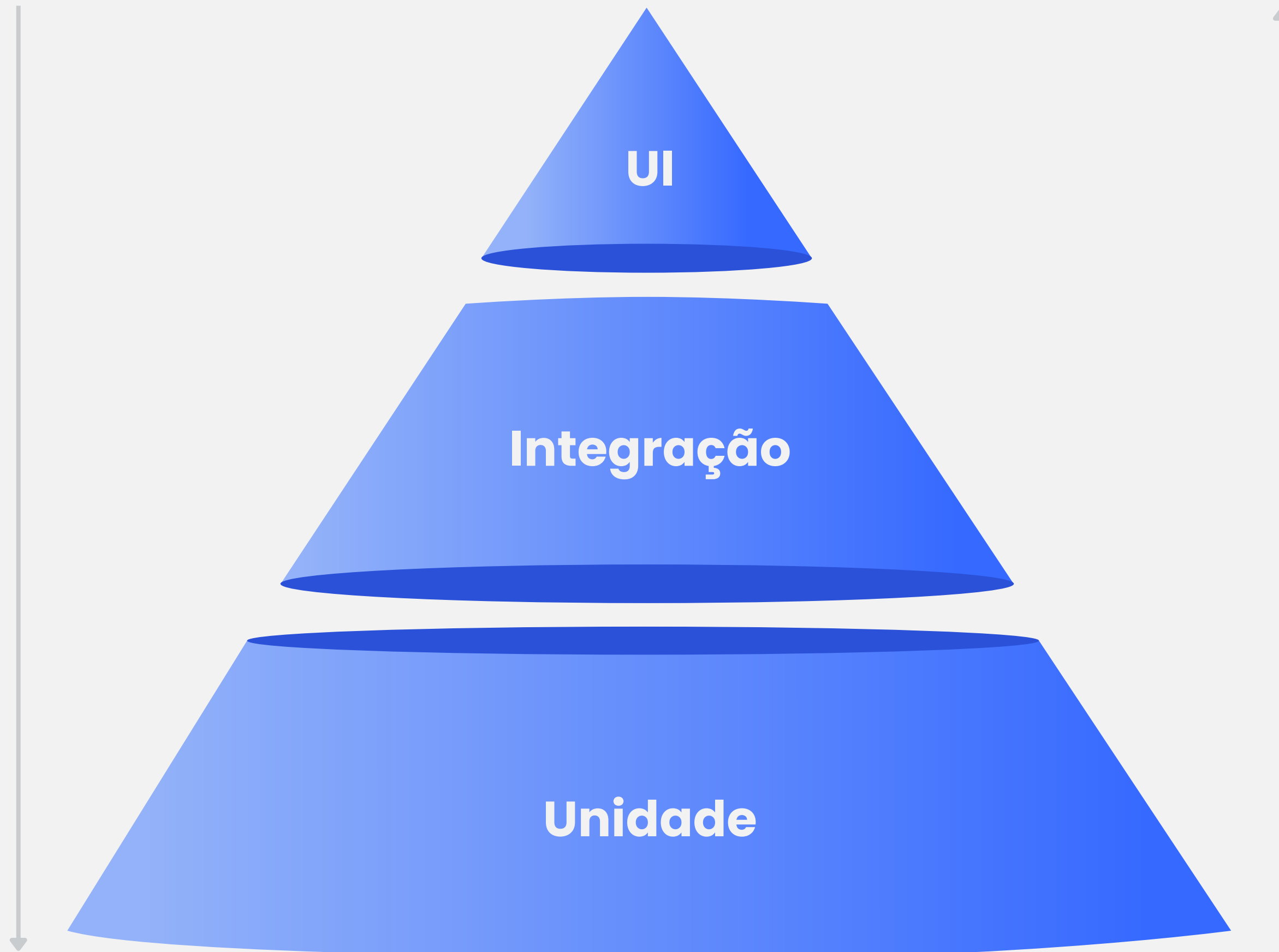
- + Qualidade aumentada
- + Tempo de lançamento reduzido
- + Menos riscos nas entregas
- + Maior confiança da equipe
- + Feedback rápido
- + Colaboração melhorada
- + Redução de custos

// Desafios

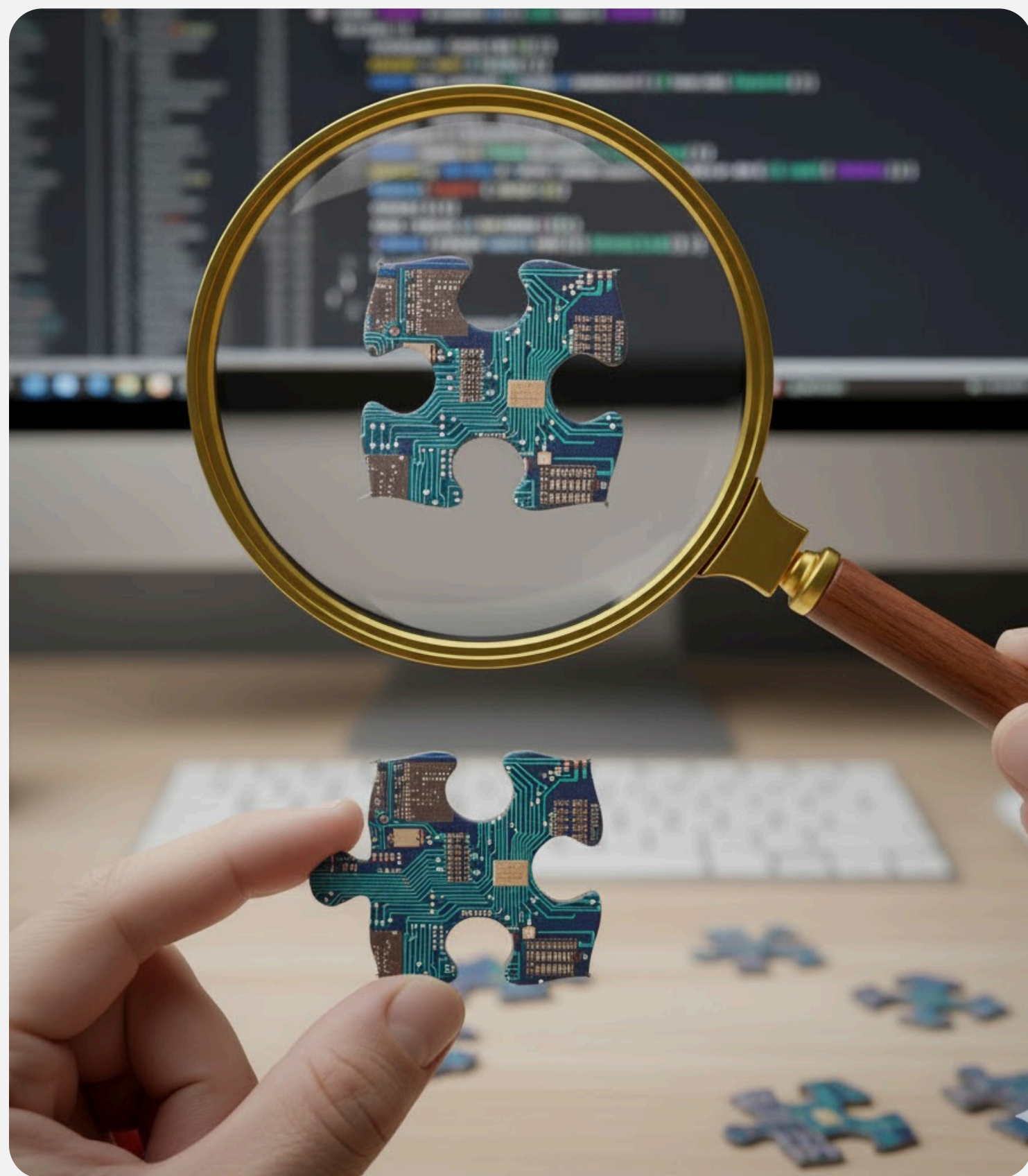
- Mudar a mentalidade da equipe
- Investimento Inicial
- Testes Abrangentes
- Manutenção do pipeline
- Gerenciamento de ambientes
- Segurança

A Pirâmide de Testes

+ Rápido
+ Específico
- Esforço
- Custo



+ Confiança
+ Cobertura
+ Lento
+ Manutenção
+ Esforço
+ Custo



O que é
Teste **Unitário**?

Teste Unitário

O quê?

Testa um único método ou classe

Como?

Ignora o resto do sistema

É o teste que foca em
UMA ÚNICA PEÇA do seu
código de forma
TOTALMENTE ISOLADA



```
1  @pytest.mark.unit
2  def test_xp_resposta_rapida():
3      assert calcular_xp(3.0) == 100
4
5
6  @pytest.mark.unit
7  def test_xp_resposta_media():
8      assert calcular_xp(10.0) == 50
```

Vantagens do Teste Unitário



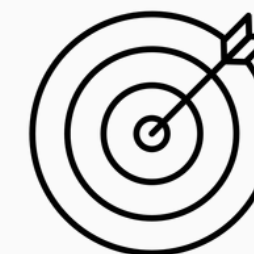
**Extremamente
Rápidos**



**Documentação
Viva**



**Rede de
Segurança**



Precisão



O que é
Teste de
Integração?

Teste de Integração

O quê?

Testa a comunicação entre
**Controller => Service => Banco de
Dados.**

É o teste que verifica se
DIFERENTES PEÇAS do seu
código **TRABALHAM BEM
JUNTAS**

Teste de Integração

Como?

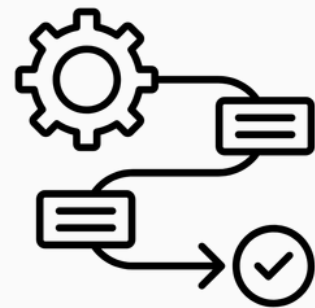
Simula um fluxo **real**, usando componentes de verdade (como um banco em memória).

É o teste que verifica se **DIFERENTES PEÇAS** do seu código **TRABALHAM BEM JUNTAS**



```
1  @pytest.mark.integration
2  def test_resposta_correta_ganha_xp(client):
3      response = client.post("/responder", data={
4          "resposta": "4",
5          "desafio_id": 1,
6          "timestamp": str(time.time()),
7      })
8      assert response.status_code == 200
9      assert "Correta" in response.text
10     assert "XP" in response.text
```

Vantagens do Teste de Integração



**Confiança no Fluxo
Completo**



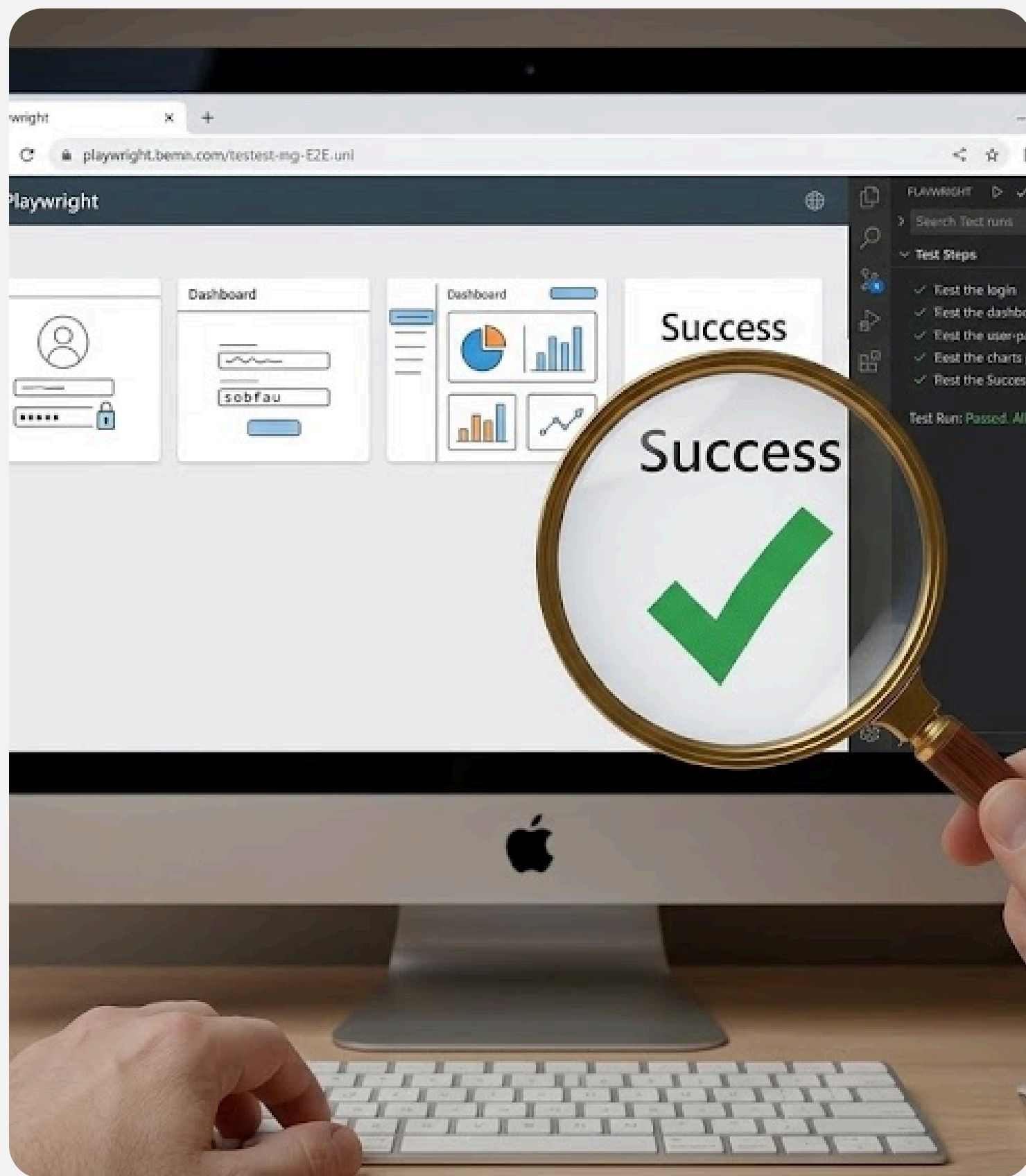
**Valida a
Interação**



**Segurança para
Refatorar**



**Encontra Bugs de
"Tradução"**



O que é
Teste de **UI (E2E)**?

Teste de UI / E2E

O quê?

Testa o sistema exatamente como o usuário final faria.

Como?

Controla um navegador real para simular um humano usando

É o teste que verifica se a **JORNADA COMPLETA** do usuário funciona do **INÍCIO AO FIM**

Conteúdo Didático e Prática

Acesse o repositório para baixar as
notas de aula e os exercícios de
fixação



<https://i-davies.github.io/conteudo-cicd-testes-automatizados/>